Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине «Математические основы интеллектуальных систем»
За 3 семестр
Тема: «Нахождение минимального остового дерева связанного неориентированного графа»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Игнатюк Д.А.

Проверил:
Козинский А.А.

Брест, 2026

Лабораторная работа №4

«Нахождение минимального остового дерева связанного неориентированного графа»

Цель работы: освоить алгоритмы Прима и Краскала для нахождения минимального остовного дерева взвешенного связного неориентированного графа и реализовать их на языке С.

Ход работы

Код

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
// ПРЕДСТАВЛЕНИЕ ГРАФА: СПИСКИ СМЕЖНОСТИ (со весами)
typedef struct Edge {
    int v;
    long long w;
    struct Edge* next;
} Edge;
typedef struct {
    int   n;
    int   m;
    Edge** adj;
} Graph;
static void add_edge(Graph* g, int u, int v, long long w)
{
    Edge* p = (Edge*)malloc(sizeof(Edge));
    p->v = v; p->w = w; p->next = g->adj[u]; g->adj[u] = p;
    p = (Edge*)malloc(sizeof(Edge));
    p->v = u; p->w = w; p->next = g->adj[v]; g->adj[v] = p;

    g->m++;
}
static int read_graph(const char* fname, Graph* g)
{
    FILE* f = fopen(fname, "r");
    if (!f) { fprintf(stderr, "ne otkryt %s\n", fname); return 0; }
    char line[256];
    int n = 0;
    int header_ok = 0;
    while (fgets(line, sizeof(line), f)) {
        if (sscanf(line, "%d", &n) == 1) { header_ok = 1; break; }
    }
    if (!header_ok || n < 0) {
        fprintf(stderr, "%s: plokhoye n\n", fname);
        fclose(f); return 0;
    }
    g->n = n;
    g->m = 0;
```

```
    g->adj = (Edge**)calloc(n + 1, sizeof(Edge*));
    if (!g->adj) { fclose(f); return 0; }
    while (fgets(line, sizeof(line), f)) {
        int u = 0, v = 0;
        long long w = 0;
        int got = sscanf(line, "%d %d %lld", &u, &v, &w);
        if (got < 3) continue;
        if (u < 1 || u > g->n || v < 1 || v > g->n) {
            fprintf(stderr, "%s: rebro (%d,%d) vne [1..%d], propusk\n",
                fname, u, v, g->n);
            continue;
        }
        if (u == v) {
            fprintf(stderr, "%s: petlya (%d,%d), propusk\n", fname, u,
v);
            continue;
        }
        if (w <= 0) {
            fprintf(stderr, "%s: nepolozhitelnyy ves (%d,%d,%lld),
propusk\n",
                fname, u, v, w);
            continue;
        }
        add_edge(g, u, v, w);
    }
    fclose(f);
    return 1;
}
static void free_graph(Graph* g)
{
    for (int i = 1; i <= g->n; i++) {
        Edge* e = g->adj[i];
        while (e) { Edge* nx = e->next; free(e); e = nx; }
    }
    free(g->adj);
    g->adj = NULL;
    g->n = g->m = 0;
}
// список рёбер + проверка связности
typedef struct {
    int u, v;
    long long w;
} WEdge;
// Собираем рёбра один раз (u < v)
static int collect_edges(const Graph* g, WEdge* edges)
{
    int idx = 0;
    for (int u = 1; u <= g->n; u++)
        for (const Edge* e = g->adj[u]; e; e = e->next)
            if (u < e->v) {
                edges[idx].u = u;
```

```
                edges[idx].v = e->v;
                edges[idx].w = e->w;
                idx++;
            }
    return idx;
}
// Проверка связности обходом (итеративный DFS)
static int is_connected(const Graph* g, int* visited, int* stack)
{
    if (g->n == 0) return 1;
    for (int i = 1; i <= g->n; i++) visited[i] = 0;
    int top = 0;
    stack[top++] = 1;
    visited[1] = 1;
    while (top > 0) {
        int u = stack[--top];
        for (const Edge* e = g->adj[u]; e; e = e->next)
            if (!visited[e->v]) { visited[e->v] = 1; stack[top++] = e-
>v; }
    }
    for (int i = 1; i <= g->n; i++)
        if (!visited[i]) return 0;
    return 1;
}
/*  АЛГОРИТМ ПРИМА
 *  Идея: растим дерево из вершины 1.
 *  На каждом шаге добавляем вершину с минимальным ребром,
 *  ведущим из дерева наружу. */
static long long prim(const Graph* g, WEdge* result)
{
    int N = g->n;
    int* inTree = (int*)calloc(N + 1, sizeof(int));  // вершина уже в
дереве?
    long long* key = (long long*)malloc((N + 1) * sizeof(long long)); //
мин. вес ребра
    int* parent = (int*)malloc((N + 1) * sizeof(int));
    const long long INF = 1000000000000000000LL;
    for (int i = 1; i <= N; i++) { key[i] = INF; parent[i] = 0; }
    key[1] = 0;  // стартовая вершина
    long long total = 0;
    int cnt = 0; // сколько рёбер добавили
    for (int step = 0; step < N; step++) {
        // Ищем непосещённую вершину с минимальным key
        int u = -1;
        long long best = INF;
        for (int i = 1; i <= N; i++) {
            if (!inTree[i] && key[i] < best) { best = key[i]; u = i; }
        }
        if (u == -1) break;
        inTree[u] = 1;
        if (parent[u] != 0) {
```

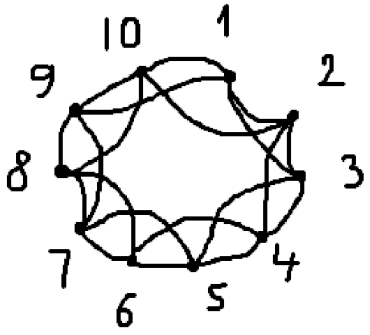
```
            result[cnt].u = parent[u];
            result[cnt].v = u;
            result[cnt].w = key[u];
            cnt++;
            total += key[u];
        }
        // Обновляем соседей
        for (const Edge* e = g->adj[u]; e; e = e->next) {
            int v = e->v;
            if (!inTree[v] && e->w < key[v]) {
                key[v] = e->w;
                parent[v] = u;
            }
        }
    }
    free(inTree); free(key); free(parent);
    return total;
}
/*  АЛГОРИТМ КРАСКАЛА
 *  Идея: сортируем рёбра по весу, добавляем в дерево,
 *  если они соединяют разные компоненты (через СНМ). */
 // СНМ — система непересекающихся множеств
static int* dsu_parent;
static int dsu_find(int x)
{
    while (dsu_parent[x] != x) {
        dsu_parent[x] = dsu_parent[dsu_parent[x]];  /* сжатие пути */
        x = dsu_parent[x];
    }
    return x;
}
static void dsu_union(int a, int b)
{
    a = dsu_find(a);
    b = dsu_find(b);
    if (a != b) dsu_parent[a] = b;
}
// Сравнение рёбер по весу — для qsort
static int cmp_edge(const void* a, const void* b)
{
    long long wa = ((const WEdge*)a)->w;
    long long wb = ((const WEdge*)b)->w;
    if (wa < wb) return -1;
    if (wa > wb) return 1;
    return 0;
}
static long long kruskal(const Graph* g, WEdge* all, int m, WEdge*
result)
{
    int N = g->n;
    // Сортируем рёбра
```

```
    qsort(all, m, sizeof(WEdge), cmp_edge);
    // СНМ: каждая вершина — своё множество
    dsu_parent = (int*)malloc((N + 1) * sizeof(int));
    for (int i = 1; i <= N; i++) dsu_parent[i] = i;
    long long total = 0;
    int cnt = 0;
    for (int i = 0; i < m; i++) {
        int u = all[i].u;
        int v = all[i].v;
        if (dsu_find(u) != dsu_find(v)) {
            dsu_union(u, v);
            result[cnt++] = all[i];
            total += all[i].w;
            if (cnt == N - 1) break;  // дерево построено
        }
    }
    free(dsu_parent);
    return total;
}
int main(int argc, char** argv)
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s <graph-file>\n", argv[0]);
        return 1;
    }
    Graph g;
    if (!read_graph(argv[1], &g)) return 1;
    printf("Fayl: %s\n", argv[1]);
    printf("Vershin: %d, ryober: %d\n\n", g.n, g.m);
    int N = g.n;
    int* visited = (int*)malloc((N + 1) * sizeof(int));
    int* buf = (int*)malloc((N + 1) * sizeof(int));
    if (!visited || !buf) {
        free(visited); free(buf); free_graph(&g); return 1;
    }
    // Проверка связности — МОД существует только для связного графа
    if (!is_connected(&g, visited, buf)) {
        printf("Graf nesvyaznyy — minimalnogo ostovnogo dereva net.\n");
        free(visited); free(buf); free_graph(&g);
        return 0;
    }
    printf("Graf svyaznyy.\n\n");
    WEdge* result_prim = (WEdge*)malloc((N) * sizeof(WEdge));
    WEdge* result_kruskal = (WEdge*)malloc((N) * sizeof(WEdge));
    WEdge* all_edges = (WEdge*)malloc((g.m + 1) * sizeof(WEdge));
    if (!result_prim || !result_kruskal || !all_edges) {
        free(visited); free(buf);
        free(result_prim); free(result_kruskal); free(all_edges);
        free_graph(&g); return 1;
    }
```

```
    int m = collect_edges(&g, all_edges);
    // ПРИМ
    printf("=== ALGORITM PRIMA ===\n");
    long long total_prim = prim(&g, result_prim);
    printf("Reobra MOD (%d shtuk):\n", N - 1);
    for (int i = 0; i < N - 1; i++) {
        printf("  %d - %d   ves %lld\n",
            result_prim[i].u, result_prim[i].v, result_prim[i].w);
    }
    printf("Summarnyy ves: %lld\n\n", total_prim);
    // КРАСКАЛ
    printf("=== ALGORITM KRASKALA ===\n");
    long long total_kruskal = kruskal(&g, all_edges, m, result_kruskal);
    printf("Reobra MOD (%d shtuk):\n", N - 1);
    for (int i = 0; i < N - 1; i++) {
        printf("  %d - %d   ves %lld\n",
            result_kruskal[i].u, result_kruskal[i].v,
result_kruskal[i].w);
    }
    printf("Summarnyy ves: %lld\n\n", total_kruskal);
    // Сверка
    if (total_prim != total_kruskal) {
        printf("VNIMANIYE: rezultaty Prima i Kraskala ne sovpali!\n");
    }
    else {
        printf("Rezultaty sovpali: ves MOD = %lld\n", total_prim);
    }
    free(result_prim); free(result_kruskal); free(all_edges);
    free(visited); free(buf);
    free_graph(&g);
    return 0;
}
```

Проверки:

Тест01 - cirkulyant_10vershin




```
tests/test01.txt: nepolozhitelnyy ves (3,5,0), propusk
Fayl: tests/test01.txt
Vershin: 10, ryober: 19

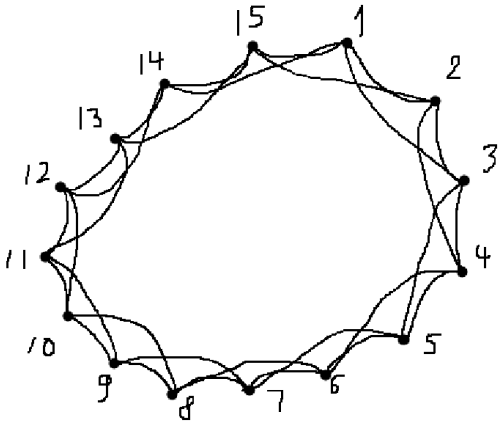
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (9 shtuk):
  1 - 10   ves 231
  10 - 8   ves 41452
  8 - 9   ves 66159
  9 - 7   ves 38
  7 - 5   ves 5290970
  5 - 6   ves 2
  6 - 4   ves 216
  4 - 3   ves 85
  3 - 2   ves 30
Summarnyy ves: 5399183

=== ALGORITM KRASKALA ===
Reobra MOD (9 shtuk):
  5 - 6   ves 2
  2 - 3   ves 30
  7 - 9   ves 38
  3 - 4   ves 85
  4 - 6   ves 216
  1 - 10   ves 231
  8 - 10   ves 41452
  8 - 9   ves 66159
  5 - 7   ves 5290970
Summarnyy ves: 5399183

Rezultaty sovpali: ves MOD = 5399183
```

Тест02 - cirkulyant_15vershin




```
Fayl: tests/test02.txt
Vershin: 15, ryober: 30

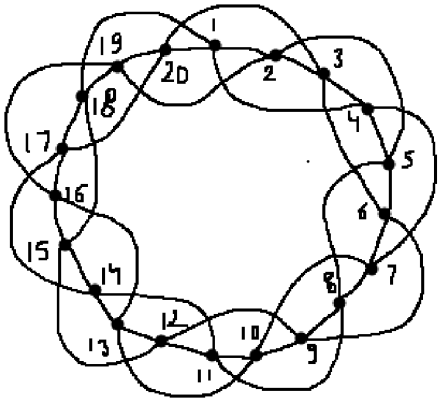
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (14 shtuk):
  1 - 2   ves 20
  2 - 4   ves 701
  4 - 5   ves 10096
  5 - 7   ves 744
  2 - 3   ves 37890
  4 - 6   ves 477641
  6 - 8   ves 98
  8 - 10   ves 92
  8 - 9   ves 541
  10 - 11   ves 173017
  11 - 12   ves 9904
  2 - 15   ves 672867
  11 - 13   ves 4757753
  13 - 14   ves 65
Summarnyy ves: 6141429
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (14 shtuk):
  1 - 2   ves 20
  13 - 14   ves 65
  8 - 10   ves 92
  6 - 8   ves 98
  8 - 9   ves 541
  2 - 4   ves 701
  5 - 7   ves 744
  11 - 12   ves 9904
  4 - 5   ves 10096
  2 - 3   ves 37890
  10 - 11   ves 173017
  4 - 6   ves 477641
  2 - 15   ves 672867
  11 - 13   ves 4757753
Summarnyy ves: 6141429

Rezultaty sovpali: ves MOD = 6141429
```

Тест03 - cirkulyant_20vershin




```
Fayl: tests/test03.txt
Vershin: 20, ryober: 40

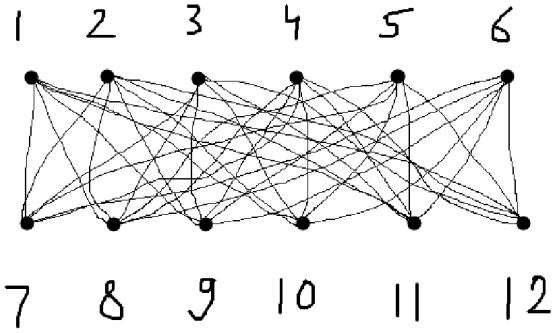
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (19 shtuk):
  1 - 2   ves 1000000000000
  2 - 3   ves 1000000000000
  1 - 4   ves 1000000000000
  2 - 5   ves 1000000000000
  3 - 6   ves 1000000000000
  4 - 7   ves 1000000000000
  5 - 8   ves 1000000000000
  6 - 9   ves 1000000000000
  7 - 10   ves 1000000000000
  8 - 11   ves 1000000000000
  9 - 12   ves 1000000000000
  10 - 13   ves 1000000000000
  11 - 14   ves 1000000000000
  12 - 15   ves 1000000000000
  13 - 16   ves 1000000000000
  14 - 17   ves 1000000000000
  1 - 18   ves 1000000000000
  2 - 19   ves 1000000000000
  1 - 20   ves 1000000000000
Summarnyy ves: 19000000000000
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (19 shtuk):
  1 - 20   ves 1000000000000
  1 - 18   ves 1000000000000
  1 - 4   ves 1000000000000
  1 - 2   ves 1000000000000
  2 - 19   ves 1000000000000
  2 - 5   ves 1000000000000
  2 - 3   ves 1000000000000
  3 - 6   ves 1000000000000
  4 - 7   ves 1000000000000
  5 - 8   ves 1000000000000
  6 - 9   ves 1000000000000
  7 - 10   ves 1000000000000
  8 - 11   ves 1000000000000
  9 - 12   ves 1000000000000
  10 - 13   ves 1000000000000
  11 - 14   ves 1000000000000
  12 - 15   ves 1000000000000
  13 - 16   ves 1000000000000
  14 - 17   ves 1000000000000
Summarnyy ves: 19000000000000

Rezultaty sovpali: ves MOD = 19000000000000
```

Тест04 - K66_12vershin




```
Fayl: tests/test04.txt
Vershin: 12, ryober: 36

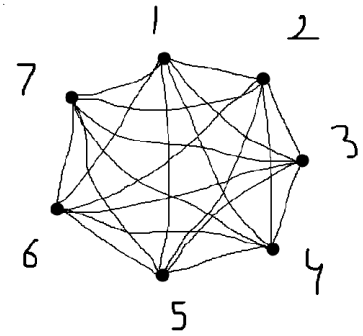
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (11 shtuk):
  1 - 7   ves 1
  1 - 8   ves 2
  1 - 9   ves 3
  1 - 10   ves 4
  1 - 11   ves 5
  1 - 12   ves 6
  7 - 2   ves 7
  7 - 3   ves 13
  7 - 4   ves 19
  7 - 5   ves 25
  7 - 6   ves 31
Summarnyy ves: 116
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (11 shtuk):
  1 - 7   ves 1
  1 - 8   ves 2
  1 - 9   ves 3
  1 - 10   ves 4
  1 - 11   ves 5
  1 - 12   ves 6
  2 - 7   ves 7
  3 - 7   ves 13
  4 - 7   ves 19
  5 - 7   ves 25
  6 - 7   ves 31
Summarnyy ves: 116

Rezultaty sovpali: ves MOD = 116
```

Тест05 - K7_7vershin




```
Fayl: tests/test05.txt
Vershin: 7, ryober: 21

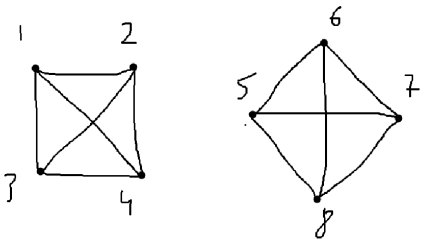
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (6 shtuk):
  1 - 2   ves 1
  2 - 3   ves 1
  2 - 4   ves 1
  2 - 5   ves 1
  2 - 6   ves 1
  2 - 7   ves 1
Summarnyy ves: 6
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (6 shtuk):
  2 - 3   ves 1
  5 - 6   ves 1
  5 - 7   ves 1
  4 - 5   ves 1
  1 - 2   ves 1
  2 - 7   ves 1
Summarnyy ves: 6

Rezultaty sovpali: ves MOD = 6
```

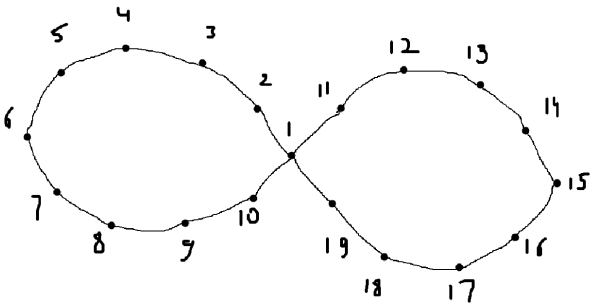
Тест06 - eyler06_dva_K4_8vershin




```
Fayl: tests/test06.txt
Vershin: 8, ryober: 12

Graf nesvyaznyy ч minimalnogo ostovnogo dereva net.
```

Тест07 - pesochnye_chasy_19vershin




```
Fayl: tests/test07.txt
Vershin: 19, ryober: 20

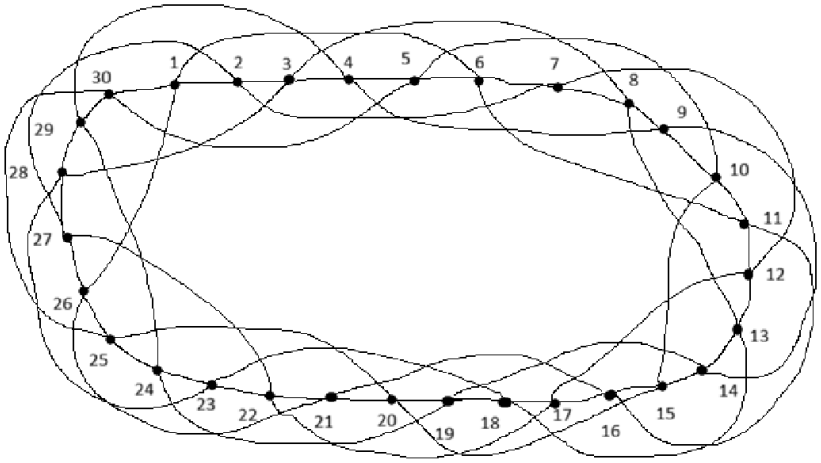
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (18 shtuk):
  1 - 2   ves 1
  2 - 3   ves 1
  3 - 4   ves 1
  4 - 5   ves 1
  5 - 6   ves 1
  6 - 7   ves 1
  7 - 8   ves 1
  8 - 9   ves 1
  9 - 10   ves 1
  1 - 19   ves 1
  19 - 18   ves 1
  18 - 17   ves 1
  17 - 16   ves 1
  16 - 15   ves 1
  15 - 14   ves 1
  14 - 13   ves 1
  13 - 12   ves 1
  12 - 11   ves 1
Summarnyy ves: 18
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (18 shtuk):
  1 - 19   ves 1
  17 - 18   ves 1
  16 - 17   ves 1
  1 - 2   ves 1
  2 - 3   ves 1
  3 - 4   ves 1
  4 - 5   ves 1
  5 - 6   ves 1
  6 - 7   ves 1
  7 - 8   ves 1
  8 - 9   ves 1
  9 - 10   ves 1
  11 - 12   ves 1
  12 - 13   ves 1
  13 - 14   ves 1
  14 - 15   ves 1
  15 - 16   ves 1
  18 - 19   ves 1
Summarnyy ves: 18

Rezultaty sovpali: ves MOD = 18
```

Тест08 – cirkulyant _30vershin




```
Fayl: tests/test08.txt
Vershin: 30, ryober: 60

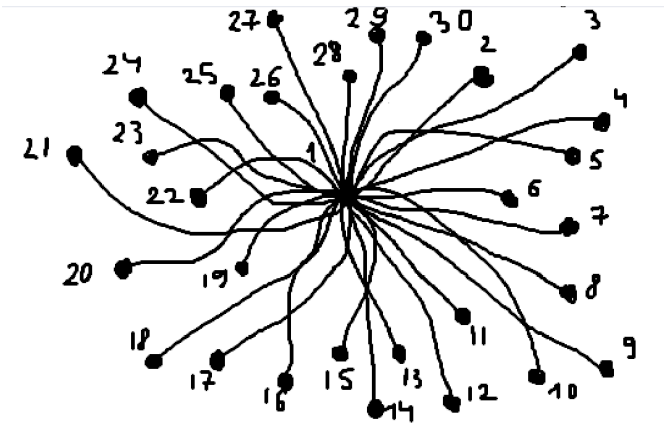
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (29 shtuk):
  1 - 2   ves 1000
  2 - 3   ves 1000
  3 - 4   ves 1000
  4 - 5   ves 1000
  1 - 6   ves 1000
  2 - 7   ves 1000
  3 - 8   ves 1000
  4 - 9   ves 1000
  5 - 10   ves 1000
  6 - 11   ves 1000
  7 - 12   ves 1000
  8 - 13   ves 1000
  9 - 14   ves 1000
  10 - 15   ves 1000
  11 - 16   ves 1000
  12 - 17   ves 1000
  13 - 18   ves 1000
  14 - 19   ves 1000
  15 - 20   ves 1000
  16 - 21   ves 1000
  17 - 22   ves 1000
  18 - 23   ves 1000
  19 - 24   ves 1000
  20 - 25   ves 1000
  1 - 26   ves 1000
  2 - 27   ves 1000
  3 - 28   ves 1000
  4 - 29   ves 1000
  1 - 30   ves 1000
Summarnyy ves: 29000
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (29 shtuk):
  1 - 30   ves 1000
  1 - 26   ves 1000
  1 - 6   ves 1000
  1 - 2   ves 1000
  2 - 27   ves 1000
  2 - 7   ves 1000
  2 - 3   ves 1000
  3 - 28   ves 1000
  3 - 8   ves 1000
  3 - 4   ves 1000
  4 - 29   ves 1000
  4 - 9   ves 1000
  4 - 5   ves 1000
  5 - 10   ves 1000
  6 - 11   ves 1000
  7 - 12   ves 1000
  8 - 13   ves 1000
  9 - 14   ves 1000
  10 - 15   ves 1000
  11 - 16   ves 1000
  12 - 17   ves 1000
  13 - 18   ves 1000
  14 - 19   ves 1000
  15 - 20   ves 1000
  16 - 21   ves 1000
  17 - 22   ves 1000
  18 - 23   ves 1000
  19 - 24   ves 1000
  20 - 25   ves 1000
Summarnyy ves: 29000

Rezultaty sovpali: ves MOD = 29000
```

Тест09 – zvezda_30vershin



```
Fayl: tests/test09.txt
Vershin: 30, ryober: 29

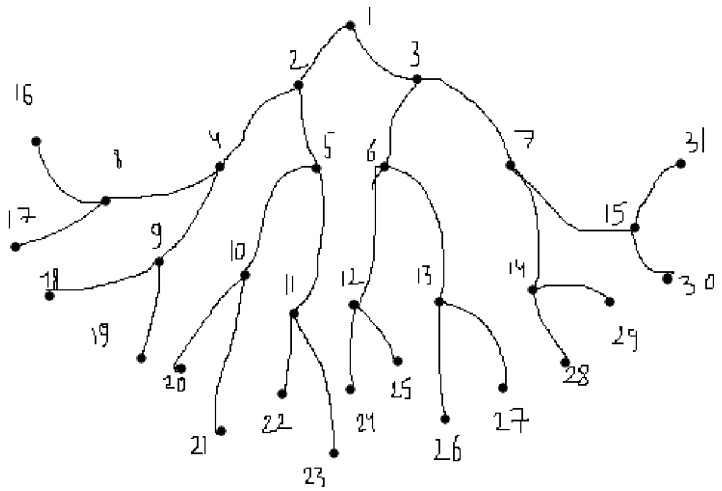
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (29 shtuk):
  1 - 10   ves 2
  1 - 11   ves 5
  1 - 30   ves 7
  1 - 22   ves 9
  1 - 3   ves 45
  1 - 12   ves 54
  1 - 17   ves 713
  1 - 4   ves 823
  1 - 7   ves 904
  1 - 6   ves 1152
  1 - 13   ves 6655
  1 - 28   ves 30932
  1 - 9   ves 95530
  1 - 19   ves 740318
  1 - 20   ves 1490370
  1 - 24   ves 9051739
  1 - 5   ves 38900975
  1 - 26   ves 56304762
  1 - 2   ves 59453716
  1 - 23   ves 415558381
  1 - 8   ves 736867156
  1 - 18   ves 852978062
  1 - 27   ves 4183409495
  1 - 14   ves 9969337655
  1 - 21   ves 39191322174
  1 - 16   ves 40218146230
  1 - 25   ves 65811281236
  1 - 29   ves 350151790144
  1 - 15   ves 419515662813
Summarnyy ves: 931212432057
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (29 shtuk):
  1 - 10   ves 2
  1 - 11   ves 5
  1 - 30   ves 7
  1 - 22   ves 9
  1 - 3   ves 45
  1 - 12   ves 54
  1 - 17   ves 713
  1 - 4   ves 823
  1 - 7   ves 904
  1 - 6   ves 1152
  1 - 13   ves 6655
  1 - 28   ves 30932
  1 - 9   ves 95530
  1 - 19   ves 740318
  1 - 20   ves 1490370
  1 - 24   ves 9051739
  1 - 5   ves 38900975
  1 - 26   ves 56304762
  1 - 2   ves 59453716
  1 - 23   ves 415558381
  1 - 8   ves 736867156
  1 - 18   ves 852978062
  1 - 27   ves 4183409495
  1 - 14   ves 9969337655
  1 - 21   ves 39191322174
  1 - 16   ves 40218146230
  1 - 25   ves 65811281236
  1 - 29   ves 350151790144
  1 - 15   ves 419515662813
Summarnyy ves: 931212432057

Rezultaty sovpali: ves MOD = 931212432057
```

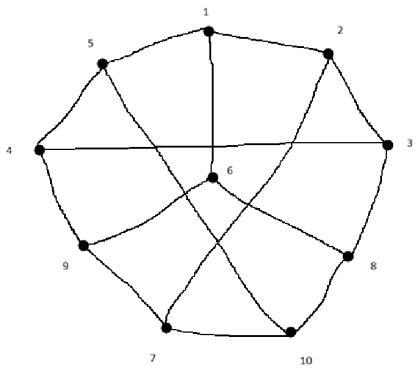
Тест10 – derevo_31vershin




```
tests/test10.txt: nepolozhitelnyy ves (2,5,0), propusk
Fayl: tests/test10.txt
Vershin: 31, ryober: 29

Graf nesvyaznyy Ч minimalnogo ostovnogo dereva net.
```

Тест11 – Petersen_10vershin



```
Fayl: tests/test11.txt
Vershin: 10, ryober: 15

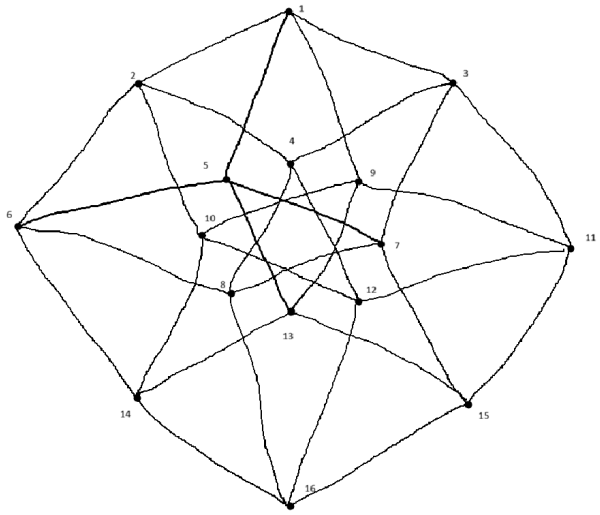
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (9 shtuk):
  1 - 6   ves 79
  6 - 8   ves 2475
  1 - 5   ves 6292047
  8 - 3   ves 78917692
  3 - 4   ves 48
  3 - 2   ves 74
  2 - 7   ves 574
  7 - 9   ves 4591549
  7 - 10   ves 27767652
Summarnyy ves: 117572190
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (9 shtuk):
  3 - 4   ves 48
  2 - 3   ves 74
  1 - 6   ves 79
  2 - 7   ves 574
  6 - 8   ves 2475
  7 - 9   ves 4591549
  1 - 5   ves 6292047
  7 - 10   ves 27767652
  3 - 8   ves 78917692
Summarnyy ves: 117572190

Rezultaty sovpali: ves MOD = 117572190
```

Тест12 – giperkub_Q4_16vershin



```
Fayl: tests/test12.txt
Vershin: 16, ryober: 32

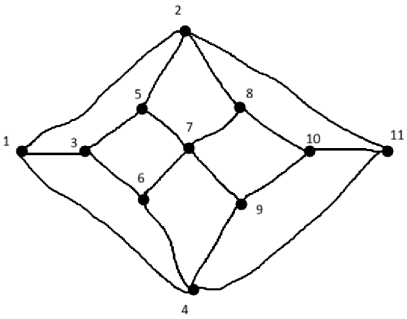
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (15 shtuk):
  1 - 2   ves 2
  1 - 3   ves 3
  1 - 5   ves 5
  2 - 4   ves 8
  1 - 9   ves 9
  2 - 6   ves 12
  2 - 10   ves 20
  3 - 7   ves 21
  4 - 8   ves 32
  3 - 11   ves 33
  4 - 12   ves 48
  5 - 13   ves 65
  6 - 14   ves 84
  7 - 15   ves 105
  8 - 16   ves 128
Summarnyy ves: 575
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (15 shtuk):
  1 - 2   ves 2
  1 - 3   ves 3
  1 - 5   ves 5
  2 - 4   ves 8
  1 - 9   ves 9
  2 - 6   ves 12
  2 - 10   ves 20
  3 - 7   ves 21
  4 - 8   ves 32
  3 - 11   ves 33
  4 - 12   ves 48
  5 - 13   ves 65
  6 - 14   ves 84
  7 - 15   ves 105
  8 - 16   ves 128
Summarnyy ves: 575

Rezultaty sovpali: ves MOD = 575
```

Тест13 – Herschel_11vershin



```
Fayl: tests/test13.txt
Vershin: 11, ryober: 18

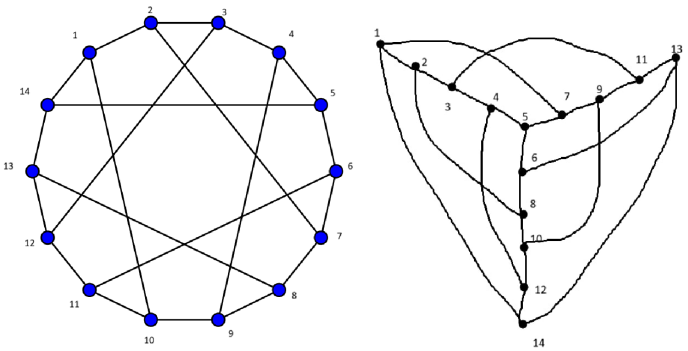
Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (10 shtuk):
  1 - 2   ves 7856523
  2 - 8   ves 89
  8 - 10   ves 93071
  10 - 9   ves 460
  9 - 7   ves 738
  2 - 5   ves 714051
  2 - 11   ves 6133464
  11 - 4   ves 77
  7 - 6   ves 74823321
  1 - 3   ves 2242106990
Summarnyy ves: 2331728784
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (10 shtuk):
  4 - 11   ves 77
  2 - 8   ves 89
  9 - 10   ves 460
  7 - 9   ves 738
  8 - 10   ves 93071
  2 - 5   ves 714051
  2 - 11   ves 6133464
  1 - 2   ves 7856523
  6 - 7   ves 74823321
  1 - 3   ves 2242106990
Summarnyy ves: 2331728784

Rezultaty sovpali: ves MOD = 2331728784
```

Тест14 – Hivud_14vershin



```
Fayl: tests/test14.txt
Vershin: 14, ryober: 21

Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (13 shtuk):
  1 - 10   ves 11
  10 - 9   ves 19
  1 - 2   ves 21
  2 - 3   ves 32
  3 - 4   ves 43
  4 - 5   ves 54
  5 - 6   ves 65
  2 - 7   ves 72
  7 - 8   ves 87
  6 - 11   ves 116
  3 - 12   ves 213
  8 - 13   ves 318
  1 - 14   ves 411
Summarnyy ves: 1462
```

```
=== ALGORITM KRASKALA ===
Reobra MOD (13 shtuk):
  1 - 10   ves 11
  9 - 10   ves 19
  1 - 2   ves 21
  2 - 3   ves 32
  3 - 4   ves 43
  4 - 5   ves 54
  5 - 6   ves 65
  2 - 7   ves 72
  7 - 8   ves 87
  6 - 11   ves 116
  3 - 12   ves 213
  8 - 13   ves 318
  1 - 14   ves 411
Summarnyy ves: 1462

Rezultaty sovpali: ves MOD = 1462
```

Тест15 – 1024vershin

```
Fayl: tests/test15.txt
Vershin: 1024, ryober: 1954

Graf svyaznyy.

=== ALGORITM PRIMA ===
Reobra MOD (1023 shtuk):
  1 - 2   ves 3
  2 - 3   ves 5
  3 - 4   ves 7
  4 - 5   ves 9
  5 - 6   ves 11
  6 - 7   ves 13
  7 - 8   ves 15
```

```
  989 - 1021   ves 2010
  990 - 1022   ves 2012
  991 - 1023   ves 2014
  992 - 1024   ves 2016
Summarnyy ves: 1017823

=== ALGORITM KRASKALA ===
Reobra MOD (1023 shtuk):
  1 - 2   ves 3
  2 - 3   ves 5
  3 - 4   ves 7
  4 - 5   ves 9
  5 - 6   ves 11
  6 - 7   ves 13
```

```
  989 - 1021   ves 2010
  990 - 1022   ves 2012
  991 - 1023   ves 2014
  992 - 1024   ves 2016
Summarnyy ves: 1017823

Rezultaty sovpali: ves MOD = 1017823
```

Вывод: Реализована программа на C для поиска минимального остовного дерева взвешенного связного неориентированного графа алгоритмами Прима и Краскала. Проверена на 15 тестах — от малых графов до графа с 1024 вершинами. Результаты совпали с теорией.